

MÓDULO 1 • PIPELINE DE DATOS Y PREPARACIÓN PROFESIONAL

Ingeniería de Características

Actividades prácticas — Feature Engineering, codificación y escalado

Propósito

Aplicar técnicas de Feature Engineering para transformar datos originales en características adecuadas para un modelo de Machine Learning.

Actividad 1. Identificar tipos de variables

Clasifica las siguientes variables como numéricas, categóricas, ordinales, temporales o de texto.

edad → _____

ingresos → _____

ciudad → _____

tipo_plan → _____

nivel_satisfaccion → _____

fecha_compra → _____

comentario_cliente → _____

cantidad_productos → _____

estado_civil → _____

Actividad 2. Analizar una variable categórica

Considera la variable:

```
tipo_plan = ['Básico', 'Premium', 'Básico', 'Estudiante', 'Premium']
```

- ¿Es una variable nominal u ordinal?
- ¿Tiene sentido asignarle directamente 0, 1 y 2?
- ¿Qué método de codificación utilizarías?
- ¿Por qué?

Actividad 3. Aplicar One-Hot Encoding

Crea un pequeño DataFrame:

```
import pandas as pd

df = pd.DataFrame({
    'ciudad': ['Montevideo', 'Canelones', 'Maldonado', 'Montevideo']
})
```

Aplica OneHotEncoder:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown='ignore')
resultado = encoder.fit_transform(df[['ciudad']])
print(resultado.toarray())
```

Responde: ¿cuántas columnas se generaron?

Actividad 4. Comparar codificaciones

Compara las siguientes situaciones:

Variable	Codificación sugerida	Justificación
Color		
Nivel educativo		
Tipo de cliente		
Nivel de satisfacción: bajo/medio/alto		

Actividad 5. Aplicar StandardScaler

Crea un DataFrame con variables de escalas diferentes:

```
df = pd.DataFrame({
    'edad': [20, 30, 40, 50],
    'ingresos': [25000, 45000, 70000, 120000]
})
```

Aplica StandardScaler y compara los valores originales con los valores transformados.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)
```

Actividad 6. Comparar métodos de escalado

Utiliza el mismo dataset y aplica:

- StandardScaler
- MinMaxScaler
- RobustScaler

Observa cómo cambia la escala de las variables.

Actividad 7. Crear una variable derivada

Supón que tienes las columnas:

```
ingresos  
cantidad_personas_hogar
```

Crea:

```
ingreso_por_persona
```

Explica por qué esta característica podría aportar información diferente de las variables originales.

Actividad 8. Crear variables temporales

Utiliza el siguiente ejemplo:

```
df['fecha'] = pd.to_datetime(df['fecha'])
```

Crea las siguientes variables:

- Año.
- Mes.
- Día.
- Día de la semana.
- Fin de semana.

Utiliza las propiedades de fecha de Pandas para realizarlo.

Actividad 9. Discretizar una variable

Tienes una variable **edad**. Crea grupos:

- 0–17: Menor.
- 18–29: Joven.
- 30–49: Adulto.
- 50 o más: Mayor.

Puedes utilizar:

```
df['grupo_edad'] = pd.cut(  
df['edad'],  
bins=[0, 17, 29, 49, 100],  
labels=['Menor', 'Joven', 'Adulto', 'Mayor']  
)
```

Actividad 10. Transformar una variable sesgada

Considera una variable de ingresos con valores muy diferentes:

```
25000, 27000, 30000, 32000, 35000, 40000, 450000
```

- Calcula la media y la mediana.
- Identifica el valor extremo.
- Aplica una transformación logarítmica.

- Compara la distribución antes y después.

Puedes utilizar:

```
df['log_ingresos'] = np.log1p(df['ingresos'])
```

Actividad 11. Crear interacciones entre variables

A partir de:

```
precio  
superficie
```

crea:

```
precio_por_m2 = precio / superficie
```

Luego propone dos nuevas características que podrían ser útiles para el mismo problema.

Actividad 12. Feature Engineering de un dataset

Selecciona un dataset que contenga al menos:

- Dos variables numéricas.
- Dos variables categóricas.
- Una variable temporal.
- Una variable objetivo.

Realiza las siguientes transformaciones:

- Codifica las variables categóricas.
- Escala las variables numéricas.
- Extrae información de la fecha.
- Crea al menos dos variables derivadas.
- Evalúa si alguna variable necesita transformación.

Actividad 13. Construir un Pipeline

Construye un pipeline que incluya al menos:

- Imputación.
- Escalado de variables numéricas.
- Codificación de variables categóricas.
- Un modelo de Machine Learning.

Utiliza Pipeline y ColumnTransformer.

```
from sklearn.compose import ColumnTransformer  
from sklearn.pipeline import Pipeline  
from sklearn.impute import SimpleImputer  
from sklearn.preprocessing import StandardScaler, OneHotEncoder  
from sklearn.linear_model import LogisticRegression  
  
preprocessor = ColumnTransformer([  
    ('num', Pipeline([  
        ('imputer', SimpleImputer(strategy='median'))],
```

```

('scaler', StandardScaler())
]), columnas_numericas),
('cat', Pipeline([
('imputer', SimpleImputer(strategy='most_frequent')),
('encoder', OneHotEncoder(handle_unknown='ignore'))
]), columnas_categoricas)
])

pipeline = Pipeline([
('preprocessor', preprocessor),
('model', LogisticRegression())
])

```

Actividad 14. Detectar Data Leakage

Analiza estas situaciones y determina si existe riesgo de data leakage.

Situación	¿Hay leakage?	Justificación
Escalar todo el dataset antes de dividirlo.		
Crear una variable usando una fecha posterior a la predicción.		
Ajustar OneHotEncoder con entrenamiento y luego transformar prueba.		
Ajustar StandardScaler solamente con X_train.		

Actividad 15. Comparar antes y después

Toma un dataset y compara el conjunto original con el conjunto después de aplicar Feature Engineering.

- Cantidad de columnas.
- Tipos de variables.
- Cantidad de variables numéricas.
- Cantidad de variables categóricas.
- Distribución de las variables.
- Valores faltantes.
- Características nuevas.

Explica qué cambios consideras beneficiosos y cuáles podrían generar problemas.

Actividad Integradora — Construcción de características

Consigna

Selecciona un problema de Machine Learning y un dataset adecuado. Realiza un proceso completo de Feature Engineering que transforme los datos originales en un conjunto de características listo para utilizar en un modelo.

El trabajo debe incluir:

- 1. Descripción del problema.
- 2. Descripción del dataset.
- 3. Identificación de los tipos de variables.
- 4. Identificación de variables categóricas.
- 5. Elección y justificación de métodos de codificación.
- 6. Escalado de variables numéricas cuando corresponda.
- 7. Creación de al menos tres nuevas características.
- 8. Al menos una transformación temporal o matemática.
- 9. Análisis de posibles variables sesgadas.
- 10. Identificación de posibles riesgos de data leakage.
- 11. Construcción de un Pipeline.
- 12. Comparación del dataset antes y después.
- 13. Justificación de las decisiones tomadas.
- 14. Conclusiones.

Autoevaluación**1. ¿Qué es Feature Engineering?**

Respuesta: Es el proceso de crear o transformar características para representar mejor un problema de Machine Learning.

2. ¿Para qué sirve One-Hot Encoding?

Respuesta: Para representar categorías mediante columnas binarias.

3. ¿Cuándo puede utilizarse Ordinal Encoding?

Respuesta: Cuando las categorías tienen un orden significativo.

4. ¿Para qué sirve StandardScaler?

Respuesta: Para estandarizar variables numéricas utilizando su media y desviación estándar.

5. ¿Cuándo puede ser útil RobustScaler?

Respuesta: Cuando existen valores atípicos que pueden afectar a otros métodos de escalado.

6. ¿Qué es una variable derivada?

Respuesta: Una variable creada a partir de una o más variables existentes.

7. ¿Qué información puede extraerse de una fecha?

Respuesta: Año, mes, día, día de semana, trimestre, hora y otras características.

8. ¿Qué es Data Leakage?

Respuesta: El uso de información que no debería estar disponible durante el entrenamiento.

9. ¿Para qué sirve ColumnTransformer?

Respuesta: Para aplicar diferentes transformaciones a distintos grupos de columnas.

10. ¿Por qué utilizar un Pipeline?

Respuesta: Para integrar y reproducir de forma consistente las transformaciones y el modelo.

Cierre

Idea clave:

Un buen modelo no depende solamente del algoritmo elegido. La forma en que los datos son representados puede ser determinante. Feature Engineering permite transformar datos disponibles en características que expresen mejor el problema y faciliten el aprendizaje del modelo.